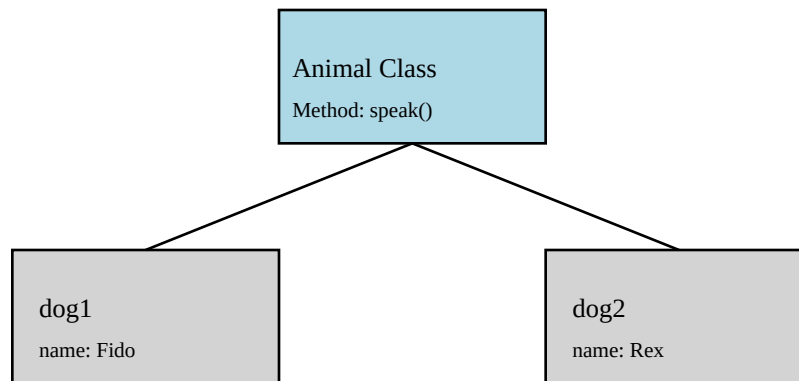


Understanding ECMAScript Inheritance (vs Class-Based OOP)

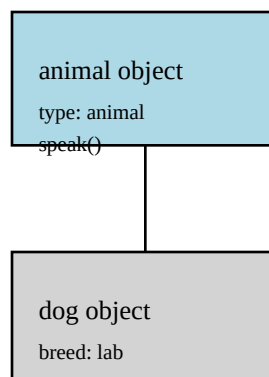
■ Part 1: Class-Based OOP (Traditional Languages like Java/C++)

In class-based languages, each instance carries its own state (like ``name``), while methods like ``speak()`` are defined once in the class and shared by all instances. Inheritance passes down structure and behavior but not actual instance data.



■■ Part 2: ECMAScript Model (Prototype-Based)

JavaScript uses objects as the fundamental building blocks. Objects can carry both state and methods, and inheritance works via prototype chains. This means objects can inherit structure, behavior, and even default state.



■ Comparison Table

Concept	Class-Based Languages	ECMAScript (JavaScript)
State location	In instances	In objects and inherited
Method location	In class (shared)	In objects and inherited
Inheritance type	Structure & behavior	Structure, behavior, & state
Mechanism	Class hierarchy	Prototype chain

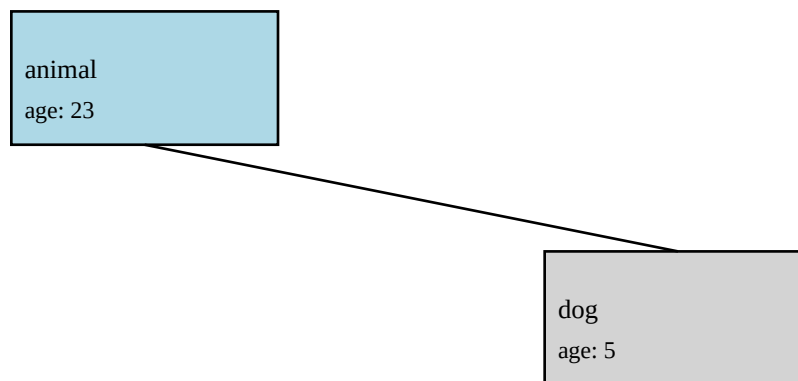
Overriding Inherited State in JavaScript

In JavaScript, when you assign a value to a property that is inherited from a prototype, it does not modify the prototype's value. Instead, it creates a new 'own' property on the object itself that shadows the inherited one.

■ Example:

```
const animal = { age: 23 }; const dog = Object.create(animal); console.log(dog.age); //  
23 (inherited) dog.age = 5; console.log(dog.age); // 5 (own property)  
console.log(animal.age); // 23 (unchanged)
```

■ Visual Model:



■ Summary Table

Action	Result
dog.age = 5	Creates own property 'age' on dog
dog.age	Reads own value, not prototype's
delete dog.age	Removes override, falls back to prototype
dog.hasOwnProperty('age')	True if dog has its own 'age'

■ Bonus Tip:

You can delete an overriding property to fall back to the inherited one.

```
delete dog.age; console.log(dog.age); // 23 (inherited again)
```